

A project report on
BUS MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE (B. Sc)

(University of Calicut)

submitted by

MUHAMMED RAASHID (Regno. FKAVSCS023)
MUHAMMED IRFAN P K (Regno. FKAVSCS011)
ATHIRA RADHAKRISHNAN M (Regno. FKAVSCS034)
HAFEEFA P M (Regno. FKAVSCS020)

Under the guidance of
AFSAL K



2023-2024

Department of Computer Science
Farook College (Autonomous)
A College with Potential for Excellence identified by UGC, New Delhi
(Affiliated to the University of Calicut)
Farook College P.O., Kozhikode, Kerala - 673 632
March 2024

A project report on
BUS MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE (B. Sc)

(University of Calicut)

submitted by

MUHAMMED RAASHID (Regno. FKAVSCS023)
MUHAMMED IRFAN P K (Regno. FKAVSCS011)
ATHIRA RADHAKRISHNAN M (Regno. FKAVSCS034)
HAFAEFA P M (Regno. FKAVSCS020)

Under the guidance of
AFSAL K



2023-2024

Department of Computer Science
Farook College (Autonomous)
A College with Potential for Excellence identified by UGC, New Delhi
(Affiliated to the University of Calicut)
Farook College P.O., Kozhikode, Kerala - 673 632
March 2024

**DEPARTMENT OF COMPUTER SCIENCE
FAROOK COLLEGE**



Certificate

This is to certify that the project work entitled **BUS MANAGEMENT SYSTEM** is a Bonafide record of the work done by **MUHAMMED RAASHID (Regno. FKA VSCS023), MUHAMMED IRFAN P K (Regno. FKA VSCS011), ATHIRA RADHAKRISHNAN M (Regno. FKA VSCS034), HAFEEFA P M (Regno. FKA VSCS020)** in partial fulfillment of the awards of degree of Bachelor of Science in Computer Science, University of Calicut, under my guidance during the year 2023-2024.

Internal Guide

Head of Department

Certified that the candidate were examined by us in the Project Viva voce, examination held on ----- and his Register Number is: -----

External Examiners:

1. -----

2. -----

Date: -----

DECLARATION

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

MUHAMMED RAASHID (Regno. FKAVSCS023)
MUHAMMED IRFAN P K (Regno. FKAVSCS011)
ATHIRA RADHAKRISHNAN M (Regno. FKAVSCS034)
HAFEEFA P M (Regno. FKAVSCS020)

Place: Farook College

Date:

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without mentioning those who made it possible, whose constant guidance and encouragement has crowned efforts with success.

First and foremost, we thank the Almighty God for being with us over the years and guiding us to the heights of success. We extend sincere gratitude to our parents for their love, care and support.

We express our sincere thanks to Principal Dr. K. A. Aysha Swapna, for giving us a chance to present our main project and providing us with all the facilities for the successful completion of the same.

We would like to express my deep sense of gratitude to Dr. V. Kabeer, Head of The Department, Department of Computer Science, for his valuable guidance and for all the help rendered during the project work and for its successful completion. We express our deep sense of gratitude to Mrs. Nusrath A, Project Coordinator for their timely checking up of the progress of our main project and in turn encouraging us to complete in time with her valuable piece of advice. We are also thankful to Mr. Afsal K, internal project guide, for his creative ideas and suggestions throughout the project.

We thank all the members of the Department of Computer Science and our friends for the timely help, advice and suggestions, which contributed either directly or indirectly to the success of the project.

MUHAMMED RAASHID (Regno. FKAVSCS023)
MUHAMMED IRFAN P K (Regno. FKAVSCS011)
ATHIRA RADHAKRISHNAN M (Regno. FKAVSCS034)
HAFEEFA P M (Regno. FKAVSCS020)

March 2024

CONTENTS

Chapter 1

1. Introduction

- 1.1. Bus Management Systems
- 1.2. Features of Existing Systems
- 1.3. Limitations of Existing Systems
- 1.4. Area and Category of the Project Work

Chapter 2

2. Problem Definition and Methodology

- 2.1. Introduction
- 2.2. Problem Definition
- 2.3. Objectives
- 2.4. Motivation
- 2.5. Methodology
- 2.6. Scope

Chapter 3

3. Analysis

- 3.1. Requirement Analysis
- 3.2. Existing System
- 3.3. Proposed System
- 3.4. Requirement Specification
 - 3.4.1. Functional Requirements
 - 3.4.2. Non-Functional Requirements
 - 3.4.3. Environmental Details
 - 3.4.4. Hardware Requirements
 - 3.4.4.1 Mobile Device Requirements
 - 3.4.5 Software Requirements
- 3.5. Feasibility Study
 - 3.5.1. Technical Feasibility
 - 3.5.2. Economic Feasibility
 - 3.5.3. Operational Feasibility

Chapter 4

4. Design

- 4.1. Introduction
- 4.2. Modularity Criteria
- 4.3. Architecture Diagrams/DFD
 - 4.3.1. DFD Level 0
 - 4.3.2. DFD Level 1
 - 4.3.3. DFD Level 2 of Process 2.0
 - 4.3.4. DFD Level 2 of Process 3.0

- 4.4. User Interference Layout
 - 4.4.1. Home Page
 - 4.4.2. Special User Page
 - 4.4.3. Admin Page
- 4.5. Database Design
 - 4.5.1. List of Entities and Attributes
 - 4.5.2. E R Diagram
 - 4.5.3. Structure of Tables

Chapter 5

5. Coding

- 5.1. Introduction
- 5.2. Flutter
- 5.3. Dart
- 5.4. Firebase
 - 5.4.1. Firestore Database
 - 5.4.2. Firebase Authentication

Chapter 6

6. System Testing

- 6.1. Introduction
- 6.2. Testing Methods
 - 6.2.1. Unit Testing
 - 6.2.2. Integration Testing
 - 6.2.3. Alpha Testing
 - 6.2.4. Beta Testing
- 6.3. Test Cases

Chapter 7

7. Conclusion

- 7.1. Future Enhancement
- 7.2. Bibliography
 - 7.2.1. Websites
 - 7.2.2. References
- 7.3. Snapshots
- 7.4. Codes

LIST OF FIGURES

Fig No.	Caption	Page No.
1	DFD Level 0	18
2	DFD Level 1	19
3	DFD Level 2 of process 2.0	19
4	DFD Level 2 of process 3.0	19
5	Home Page	20
6	Special User Page	20
7	Special User Page	21
8	Admin Page	21
9	E R Diagram	23

LIST OF TABLES

Table No.	Caption	Page No.
1	Admin	24
2	Special User	24
3	Bus Info	24
4	Invoice	24
5	Route	24
6	Schedule	24
7	Admin Sign-In Page	29
8	Special User Sign-In Page	29
9	Special User Sign-Up Page	29
10	Invoice	30

CHAPTER 1

INTRODUCTION

Introducing a revolutionary bus travel app designed for both passengers and bus employees. Our app enables users to effortlessly plan their trips in advance by viewing bus schedules and tracking the real-time location of their bus. For bus employees, such as conductors, the app provides tools to manage schedules, update bus statuses, and automate the invoicing process. With special privileges for conductors, the app ensures a seamless experience for all users. Join us and experience a new era of convenience in bus travel.

1.1 Bus Management System

The application is designed to enhance the experience of bus travelers and employees, particularly conductors. Users can easily plan their trips by accessing bus schedules and tracking the real-time location of their bus. Conductors can manage bus schedules, update statuses, and automate the invoicing process. The app ensures safety and convenience, allowing conductors to calculate, update, and review invoices efficiently. Owners can receive updated invoices instantly and correct any mistakes promptly. With a user-friendly interface and special privileges for conductors, our app revolutionizes bus travel for all users.

1.2 Features of Existing Systems

- Improved the bus ticketing process.
- Bus schedule listed.
- Bus operators get reliable, cost-effective and transparent bus operations.

1.3 Limitations of Existing Systems

- Bus availability is not mentioned.
- Difficult to add new info or edit existing info.
- Lack of features such as invoice management.

1.4 Area and Category of the Project Work

The project work for the bus management app encompasses several key areas and categories, including information technology (IT), transportation management, mobile application development, data management, user experience (UX) design, operations management, and software engineering.

CHAPTER 2

PROBLEM DEFINITION & METHODOLOGY

2.1 Introduction

In today's fast-paced world, efficient transportation systems are crucial. However, the bus transportation sector often faces challenges in terms of operational efficiency, passenger convenience, and administrative tasks. To address these issues, we have developed an innovative bus management application that caters to the needs of both passengers and bus employees, particularly conductors. This app aims to streamline trip planning, enhance bus tracking capabilities, and automate invoicing processes, ultimately improving the overall bus travel experience.

2.2 Problem Definition

The challenges faced by the bus transportation sector in India are complex and multifaceted, often resulting in discomfort and inconvenience for passengers. Overcrowding is a common issue, with buses frequently filled beyond capacity, compromising passenger safety and comfort. Additionally, buses often fail to adhere to fixed schedules, leading to uncertainty and inconvenience for passengers and increasing stress for conductors who struggle to update their records in time. Insufficient seating capacity further exacerbates the problem, forcing passengers to stand or endure uncomfortable conditions. Lack of adequate information about bus routes, timings, and stops adds to the challenge, especially for passengers unfamiliar with the area. The use of paper records for invoicing presents its own set of problems, including bad data integrity due to the difficulty of writing records on a soft surface while the bus is in motion. Moreover, the inadequate updating of records in time increases the likelihood of conductors forgetting to update their invoices or duplicating entries, further compromising data integrity. Human error is also a significant concern, as manual calculations for invoices are prone to mistakes, affecting both the accuracy of records and the efficiency of the invoicing process. Addressing these challenges requires innovative solutions that enhance operational efficiency, improve passenger experience, and ensure the safety and comfort of all bus travelers.

2.3 Objectives

- Time management
- Reduced waiting time for passengers
- Avoid missing buses
- Real-time updates
- Better time management
- Time saving
- Reduced human error

2.4 Motivation

This application is motivated by the need to address key challenges in the bus transportation sector, such as overcrowding, lack of punctuality, and insufficient seating capacity. By leveraging technology, we aim to enhance the overall travel experience for both passengers and bus employees. Through real-time bus tracking, up-to-date schedules, and automated invoicing, our app seeks to improve efficiency, comfort, and safety. Beyond addressing immediate challenges, we aspire to drive innovation and progress in the industry, ultimately creating a more efficient and enjoyable bus travel experience for all.

2.5 Methodology

This app's methodology is rooted in thorough research, precise design, and continuous improvement. We began by researching the challenges faced by bus travelers and employees, informing us about our user-friendly interface design. We prioritized user feedback, conducted extensive testing, and integrated features like real-time bus tracking and automated invoicing. Post-deployment, we continue to maintain and support the app, incorporating user feedback and technological advancements. We provide training for bus employees, aiming to enhance the bus travel experience and operational efficiency. Our goal is to set a new standard for bus management systems through ongoing iteration and excellence.

2.6 Scope

In the real world, the scope of this app would have a significant impact on the bus transportation sector. It would revolutionize the way passengers plan and experience their bus journeys, as well as streamline operations for bus employees. For passengers, the app would provide a user-friendly interface to easily access bus schedules, track bus locations in real-time, and receive updates on routes and stops. This would result in more efficient trip planning, reduced waiting times, and an overall improved travel experience. For bus employees, such as conductors, the app would automate many manual tasks, such as managing schedules and updating bus statuses. This would increase operational efficiency, reduce errors, and improve overall productivity. The automated invoicing feature would also simplify the process of calculating and updating invoices, saving time and reducing stress for conductors. Overall, the app would enhance the efficiency, convenience, and safety of bus travel for both passengers and employees, leading to a more reliable and enjoyable experience for all stakeholders involved in the bus transportation sector.

CHAPTER 3

ANALYSIS

3.1 Requirement Analysis

The requirement analysis is the process of identifying system requirements through the observation of the current system, discussion with the potential users etc. A proper analysis reduces the ambiguity and effort in developing the system.

3.2 Existing System

The Chalo app is an existing bus management system that provides real-time bus tracking, route information, and other features to improve the bus travel experience. The app allows users to track buses on a map, plan their journeys, and receive alerts about bus arrivals and departures. It also provides bus operators with tools to manage their fleets, monitor bus performance, and communicate with passengers. Overall, the Chalo app aims to make bus travel more efficient, convenient, and enjoyable for passengers and bus operators alike.

3.3 Proposed System

The proposed system is the app we have been discussing, which aims to address the shortcomings of existing bus management systems by providing a comprehensive solution for both passengers and bus employees. This app will offer features such as real-time bus tracking, schedule management, automated invoicing, and improved communication tools. The goal of the app is to enhance the overall bus travel experience by making it more efficient, convenient, and user-friendly for all stakeholders.

3.4 Requirement Specification

The online Event Recording System provides an easy interface to record and access the various events happening in the college. The functional and non-functional requirements of the proposed system are underlined below.

3.4.1 Functional Requirements

- User authentication
- Bus details
- Live Tracking

3.4.2 Non-functional Requirements

- Scalable
- Easy to use UI
- Password based authentication
- Security

3.4.3 Environment Details

- Tool : Flutter, VS Code.
- Language : Dart.
- Operating System : Windows/Linux.

3.4.4 Hardware Requirements

- Processor : Intel Core i5
- Memory : 4GB RAM or above.
- Hard disk : 120GB minimum or above.

3.4.4.1 Mobile Device Requirements

- Operating System : Android 4.1 and above
- Memory : 4GB RAM
- Storage : 600MB minimum

3.4.5 Software Requirements

- Back End : Firebase
- Front End : Flutter, Dart
- Platform : Android

3.5 Feasibility Study

3.5.1 Technical feasibility

The technologies used in this project are Flutter as front-end and firebase as back-end. The technologies are freely available and technical skills are easily manageable. Time limitations of the product development and ease of implementing using these technologies are synchronized. Therefore, the project is technically feasible.

3.5.2 Economical feasibility

The system will follow a freeware software standard. No charges will be charged from the users. The development cost for the software will also be negligible. Therefore, the project is economically feasible.

3.5.3 Operational feasibility

The performance of the application will not degrade with the increase of potential users. Firebase will ensure to provide an adequate speed for database transactions. No additional training should be required to use the system. Any person with minimum knowledge about the usage of mobile phones can efficiently use the application. Therefore, the project is operational feasible.

CHAPTER 4

DESIGN

4.1 Introduction

The phase translates the requirement collected in the earlier stage to a detailed design of the system.

4.2 Modularity Criteria

This app is divided into three parts. Functionalities vary depending upon the user level.

- Registration, admins can add SP Users, and Bus instances
- Searching, normal users can search for the bus based on the 'to' and 'from' information.
- Tracking, normal users can look at their location or look at the location of an available bus.
- Availability, registered SP users can toggle bus availability 'true' or 'false'.
- Adding data, SP users can add invoice information to the app.

4.3 Architecture Diagrams/DFD

4.3.1 DFD Level 0

Level-0

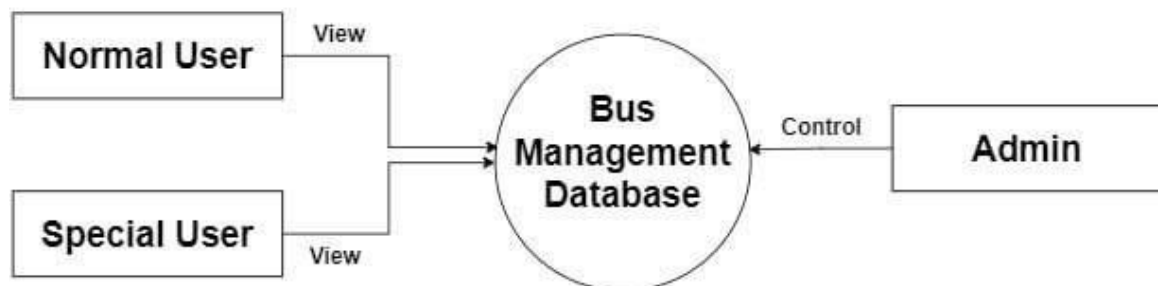


Figure 1

4.3.2 DFD Level 1

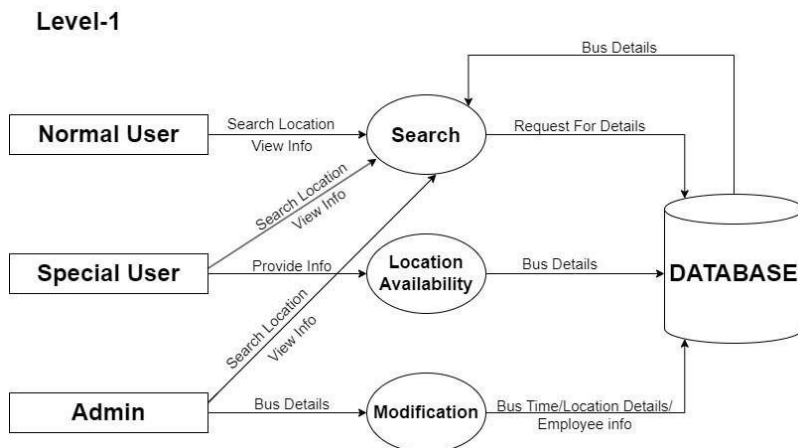


Figure 2

4.3.3 DFD Level 2 of process 2.0

Level 2 of 2.0

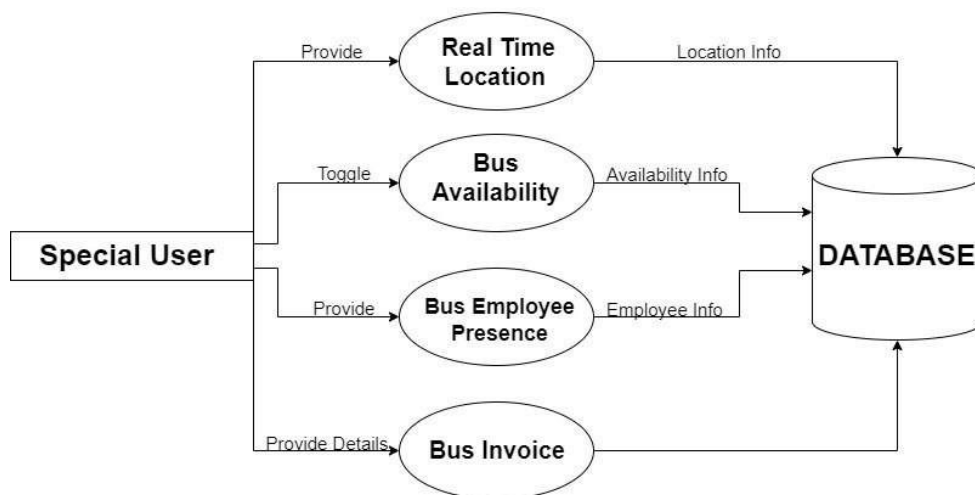


Figure 3

4.3.4 DFD Level 2 of process 3.0

Level 2 of 3.0

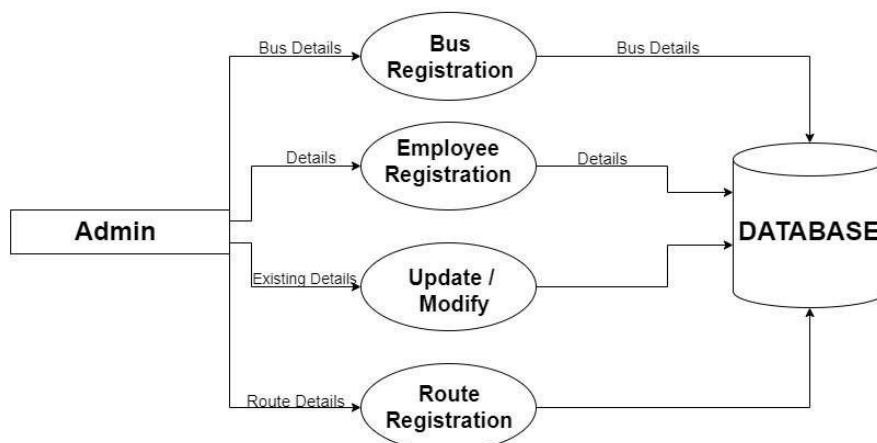


Figure 4

4.4 User Interface Layout

User interfaces are very simple to ensure a better user experience. Here we include some samples of the user interface layout.

4.4.1 Home Page

This interface shows the user home page. It depicts the following process:
Skip as a normal user,

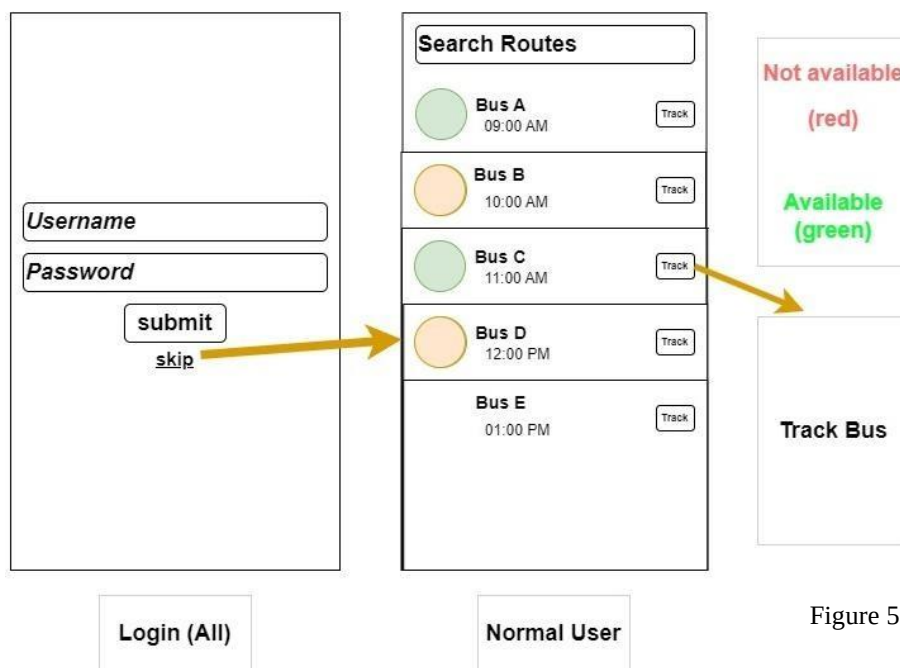


Figure 5

4.4.2 Special User Page

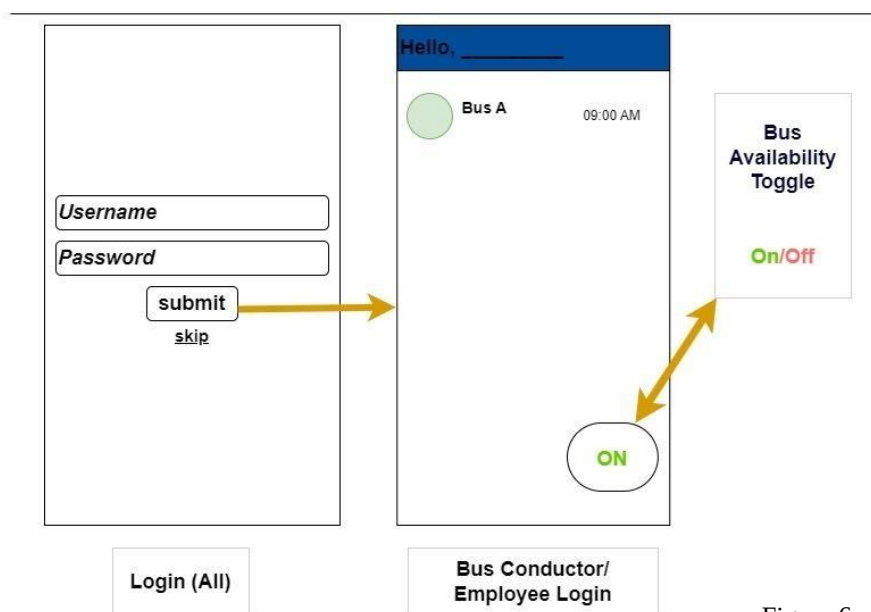


Figure 6

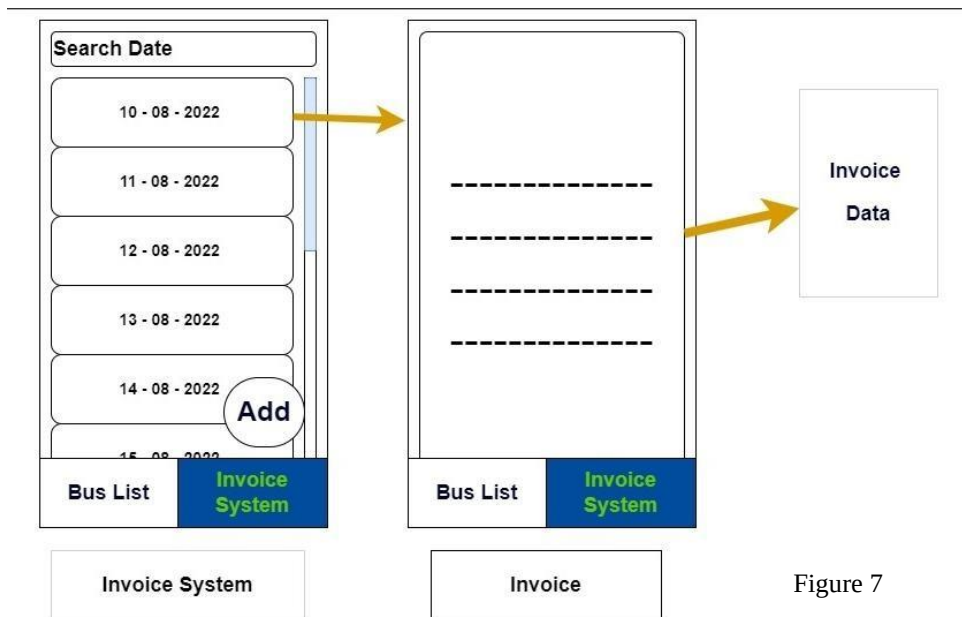


Figure 7

4.4.3 Admin Page

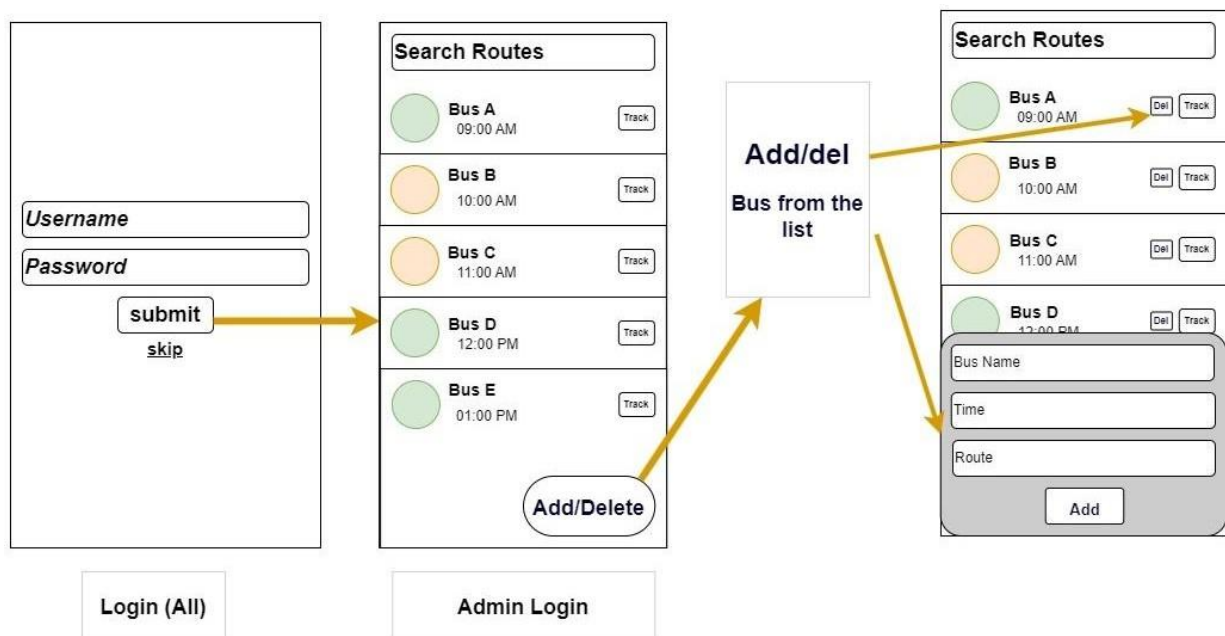


Figure 8

4.5 Database Design

4.5.1 List of Entities and Attributes

Bus Info

- Bus name
- Bus Id
- Employees
- Bus Availability

Admin

- Username
- Password
- Admin Id

Special User

- Username
- Password
- User Id

Invoice

- Invoice Id
- Total Payment
- Date

Schedule

- Schedule Id
- Arrival Time
- Departure Time

Route

- Route Id
- From
- To

4.5.2 E R Diagram

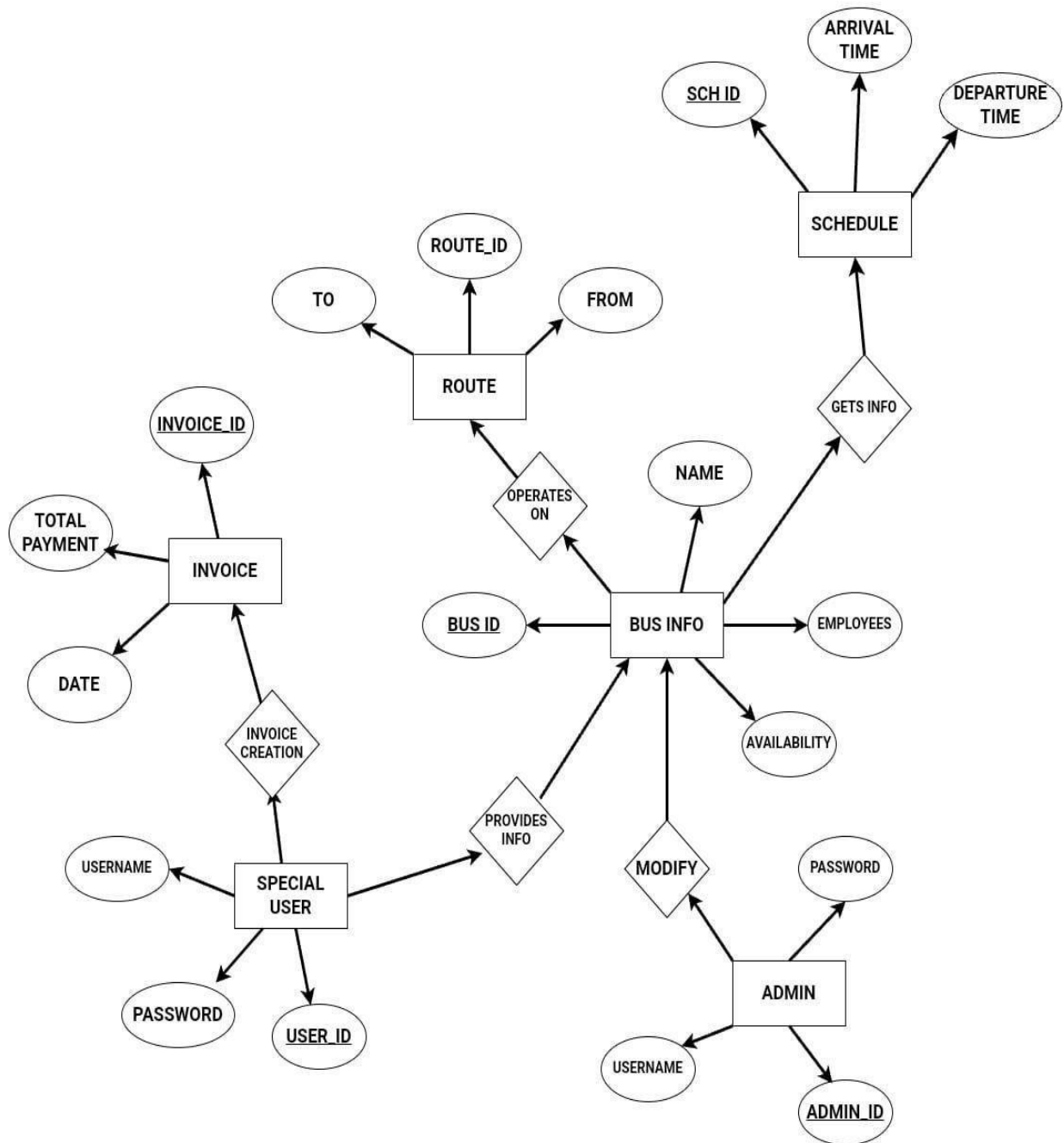


Figure 9

4.5.3 Structure of Tables

ADMIN			
Attribute	Data type	Size	Constraints
Username	VARCHAR	15	NOT NULL
Password	VARCHAR	20	NOT NULL
ADMIN_ID	VARCHAR	15	NOT NULL

Table 1

SPECIAL USER			
Attribute	Data type	Size	Constraints
Username	VARCHAR	15	NOT NULL
Password	VARCHAR	20	NOT NULL
USER_ID	VARCHAR	15	NOT NULL

Table 2

BUS INFO			
Attribute	Data type	Size	Constraints
BUS_ID	VARCHAR	15	NOT NULL
Name	VARCHAR	20	NOT NULL
Employees	VARCHAR	15	NOT NULL
Availability	BOOLEAN	15	NOT NULL

Table 3

INVOICE			
Attribute	Data type	Size	Constraints
INVOICE_ID	VARCHAR	15	NOT NULL
Date	DATE	20	NOT NULL
Total Payment	INT	15	NOT NULL

Table 4

ROUTE			
Attribute	Data type	Size	Constraints
To	VARCHAR	15	NOT NULL
From	VARCHAR	20	NOT NULL
ROUTE_ID	VARCHAR	15	NOT NULL

Table 5

SCHEDULE			
Attribute	Data type	Size	Constraints
SCH_ID	VARCHAR	15	NOT NULL
Arrival Time	VARCHAR	15	NOT NULL
Departure Time	VARCHAR	15	NOT NULL

Table 6

CHAPTER 5

CODING

5.1 Introduction

The coding phase is a vital step in creating software. It follows the planning and design stage, where developers map out how the software will function and appear. During coding, they write, test, and refine the code to ensure the software behaves as intended. This phase demands significant time and attention to detail, as well as a deep understanding of programming languages and tools. The quality of the code produced here is paramount, as it forms the foundation for testing and deploying the software.

5.2 Flutter

Flutter is a mobile application development framework developed by Google. It allows developers to build high-performance and visually appealing applications for both Android and iOS platforms using a single codebase. Flutter uses the Dart programming language, which offers a range of features that make it easy to build scalable and performant applications. One of the significant advantages of Flutter is its fast and efficient Hot Reload feature, which allows developers to see changes in the app's UI and functionality in real-time. Flutter's widget-based architecture also makes it easy to create custom user interfaces that can be reused across multiple platforms. This feature saves developers time and effort and allows them to focus on building core features rather than spending time on creating UI components from scratch. Flutter has gained popularity among mobile app developers due to its ease of use, fast development cycles, and ability to build high-quality applications.

5.3 Dart

Dart is an open-source programming language developed by Google that is used to build web, mobile, and desktop applications. With its fast and efficient runtime, object-oriented structure, and integration with the Flutter framework, it has become increasingly popular among developers for building high-performance and visually appealing mobile applications. Its support for asynchronous programming and compilation to native code and JavaScript also makes it a suitable choice for building server-side applications. Overall, Dart

offers a range of features that make it a powerful and efficient tool for modern application development.

5.4 Firebase

Firebase is a mobile and web application development platform that provides developers with a comprehensive set of tools and services to build scalable and high-performance applications. Developed by Google, Firebase offers real-time database, authentication, hosting, and analytics services, among others, allowing developers to easily integrate its services into their applications using simple APIs and SDKs. With its robust security features, flexible pricing options, and active community, Firebase is a popular choice for app development projects of all sizes and complexity levels. Its ease of use and seamless data synchronization capabilities make it an ideal platform for building dynamic and interactive applications.

5.4.1 Firestore Database

Firestore is a flexible, scalable, and cloud-based NoSQL database offered by Firebase. It is designed to store and synchronize data in real-time across multiple devices and platforms, making it ideal for building dynamic and collaborative applications. Firestore supports automatic scaling and provides powerful querying and indexing capabilities, enabling developers to manage and retrieve large amounts of data quickly and efficiently. It also offers seamless integration with other Firebase services such as authentication, cloud storage, and hosting, allowing developers to build end-to-end solutions with ease.

5.4.2 Firebase Authentication

Firebase Authentication is a secure and easy-to-use authentication service provided by Firebase. It allows developers to add user authentication and authorization capabilities to their applications, enabling users to sign up, sign in, and manage their account information. Firebase Authentication supports various authentication methods, including email/password, phone number, social media accounts, and custom authentication mechanisms, making it flexible and versatile.

CHAPTER 6

SYSTEM TESTING

6.1 Introduction

System testing is carried out in a planned manner according to the system test plan document. The system test plan identifies all testing-related activities that must be performed, specifies the schedule of testing, and allocates resources. It also lists all the test cases and the expected outputs for each test case. It ensures that the system works accurately and efficiently, before live operation commences. It helps in the noting and correction of errors.

6.2 Testing Methods

6.2.1 Unit Testing

Unit testing focusses on verification efforts made on the smallest unit of software design, a module. The modules are tested separately, and this testing is carried out during the programming stage itself. The project can be divided into modules: Contract management, Task scheduling, Data management, Reporting and Feedback. Each of these modules contains multiple forms, which have been individually tested. The module interface is tested to ensure that information properly flows in and out of the program unit under test. It is ensured that each module works satisfactorily regarding the output expected from it.

6.2.2 Integration Testing

Integration testing is used to take the unit tested modules and build a program structure. All the modules are combined and then tested as a whole. Data can be lost across interfaces and one module may have an adverse effect on others. Moreover, sub functions when combined may not produce the desired major functions. All these make integration testing necessary. Here, error correction is difficult because their isolation from the entire program becomes complicated.

6.2.3 Alpha Testing

Alpha testing takes place at the developer's side by the internal teams, before release to the external customers. Developers observe the working of the application and note problems. Generally, this testing is performed without the involvement of the development teams. It is done when development is about to be completed. Minor design changes can still be made because of alpha testing. The focus of this testing is to simulate real users by using black box and white box techniques and the aim is to carry out the tasks that a typical user might perform, diagnose any errors that arise and rectify them.

6.2.4 Beta Testing

Beta testing of a product is performed by the real users of the software application in a real environment and can be considered as a form of external user acceptance testing. The beta version of the software is released to a limited number of end-users of the product to obtain feedback on the product quality. It reduces the risk of product failure and provides increased quality of the product through customer validation. It is the final test before shipping a product to the customers. Direct feedback from customers is a major advantage of this testing in addition to helping to test the product in a real-time environment. It creates goodwill with the customers and increases customer satisfaction.

Since this project has been developed using an iterative model, it has been monitored by the client, Eagle Environmental service and pest control establishment, periodically to notify the desired changes to be made. This served as customer validation and thereby helped in creating a better solution for the clients.

6.3 Test Cases

A test case in software engineering is a set of conditions or variables under which a tester will determine whether an application or software system is working correctly or not. The test cases in our project context are:

- **Admin Sign-in Page**

Input	Expected Result	Actual result	Remarks
Invalid username and password	Unsuccessful login	Unsuccessful login	Pass
Valid username, invalid password	Unsuccessful login	Unsuccessful login	Pass
Invalid username, valid password	Unsuccessful login	Unsuccessful login	Pass
Valid username and password	Successful login	Successful login	Pass

Table 7

- **Special User Sign-in Page**

Input	Expected Result	Actual result	Remarks
Invalid username and password	Unsuccessful login	Unsuccessful login	Pass
Valid username, invalid password	Unsuccessful login	Unsuccessful login	Pass
Invalid username, valid password	Unsuccessful login	Unsuccessful login	Pass
Valid username and password	Successful login	Successful login	Pass

Table 8

- **Special User Sign-up Page**

Input	Expected Result	Actual Result	Remarks
Blank Entry	Unsuccessful login	Unsuccessful login	Pass
Empty Name	Unsuccessful login	Unsuccessful login	Pass
Empty Registration No	Unsuccessful login	Unsuccessful login	Pass
Empty Username	Unsuccessful login	Unsuccessful login	Pass
Empty Email	Unsuccessful login	Unsuccessful login	Pass
Empty Password	Unsuccessful login	Unsuccessful login	Pass
Empty Route A	Unsuccessful login	Unsuccessful login	Pass
Empty Route B	Unsuccessful login	Unsuccessful login	Pass

Table 9

- **Invoice**

Input	Expected Result	Actual Result	Remarks
Blank Entry	Invoice adding Unsuccessful	Invoice adding Unsuccessful	Pass
Empty Trip No	Invoice adding Unsuccessful	Invoice adding Unsuccessful	Pass
Empty From	Invoice adding Unsuccessful	Invoice adding Unsuccessful	Pass
Empty To	Invoice adding Unsuccessful	Invoice adding Unsuccessful	Pass
Empty Total Tickets	Invoice adding Unsuccessful	Invoice adding Unsuccessful	Pass
Empty Remaining Tickets	Invoice adding Unsuccessful	Invoice adding Unsuccessful	Pass
Empty Price	Invoice adding Unsuccessful	Invoice adding Unsuccessful	Pass

Table 10

CHAPTER 7

CONCLUSION

The project entitled "APP NAME" was successfully completed. The system is developed with much care, and free of errors, and at the same time it is efficient and less time-consuming. The purpose of this project was a comprehensive solution for both bus travelers and employees, enhancing the convenience and efficiency of bus transportation. By providing access to bus schedules, real-time tracking, and facilitating seamless trip planning, the app caters to the needs of users from diverse backgrounds, including students, adults, and bus conductors. With features empowering conductors to manage schedules, update routes, and streamline invoicing processes, the app not only improves operational efficiency but also enhances user experience. Through a user-friendly interface and specialized privileges for conductors, the app aims to revolutionize the bus transportation experience, making it safer, more reliable, and more accessible for all stakeholders involved.

7.1 Future Enhancement

- App in background tracking
- Robust bus instance system
- Adding polylines to the map
- Integration with Smart City Infrastructure

7.2 Bibliography

7.2.1 Websites

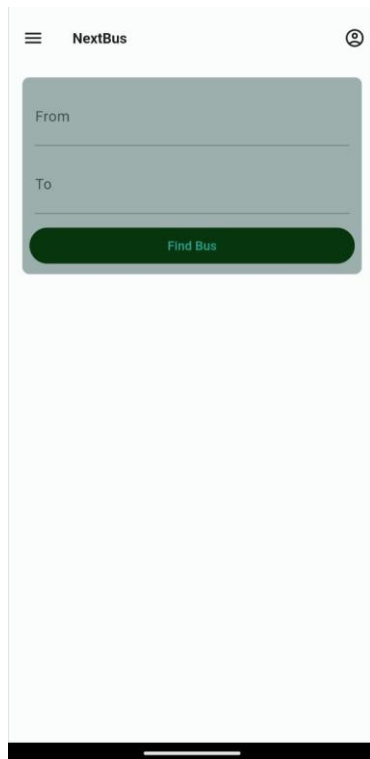
- GitHub - <https://github.com>
- Flutter gem - <https://fluttergems.dev>
- Stack overflow - <https://stackoverflow.com>
- Pub.dev - <https://pub.dev>
- Google Gemini - <https://gemini.google.com>
- Udemy - <https://www.udemy.com>
- ChatGPT - <https://chat.openai.com>

7.2.2 References

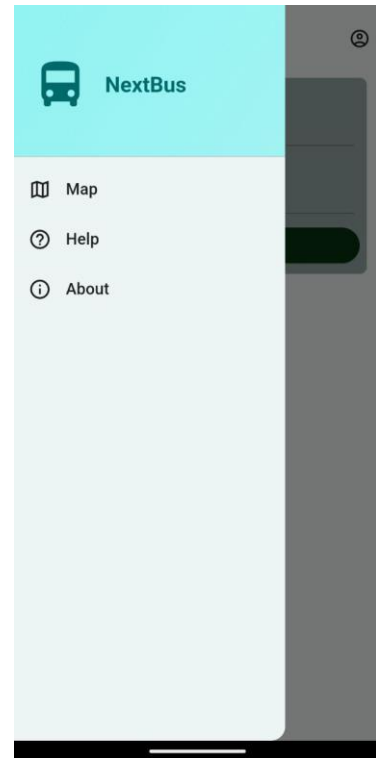
- Flutter & Dart - The Complete Guide [2024 Edition] by Maximilian Schwarz muller
- Flutter & Firebase Tutorial: Build 5 Social Media Apps by Rivaan Ranawat
- Flutter Malayalam Tutorial Series by Brototype Malayalam
(https://www.youtube.com/watch?v=M3ni_jDqY6E&list=PLY-ecO2csVHcUIBVvIMAA3dbja12TFJiN)
- Code related doubts cleared by <https://stackoverflow.com/>
- A.I assistant - Google Gemini (<https://gemini.google.com/app>)
- Chalo App – Existing App

SNAPSHOTS

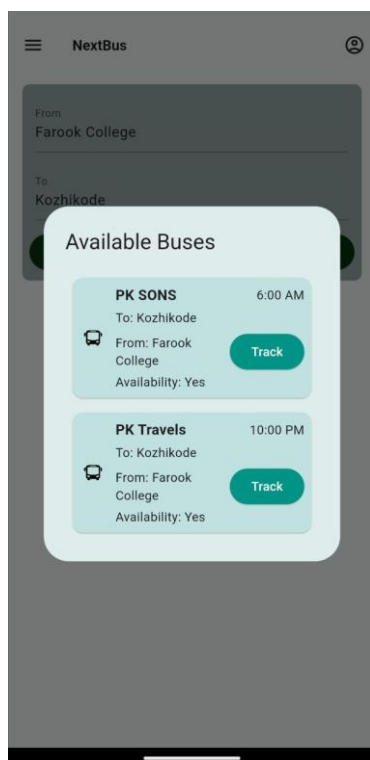
Home Page



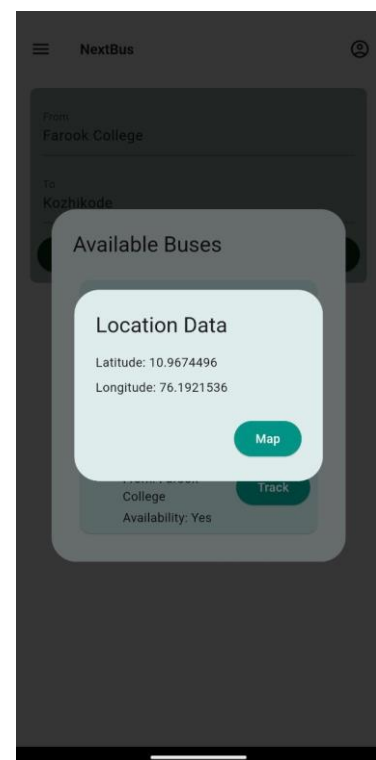
Menu Screen



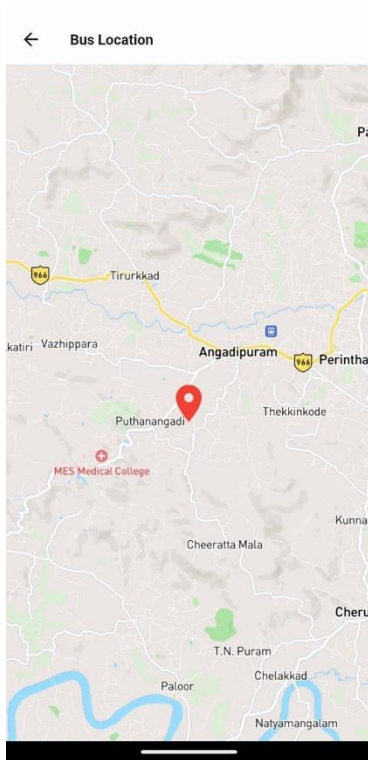
Availability of Bus



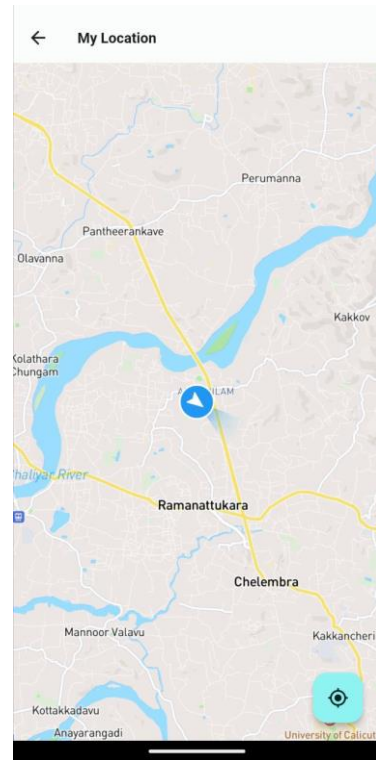
Location of Bus



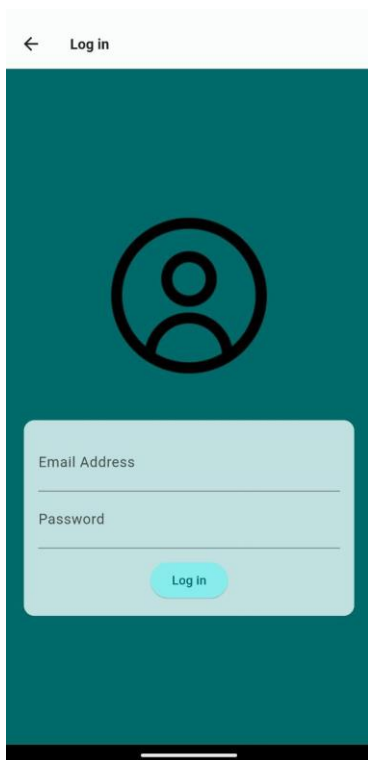
Bus Location



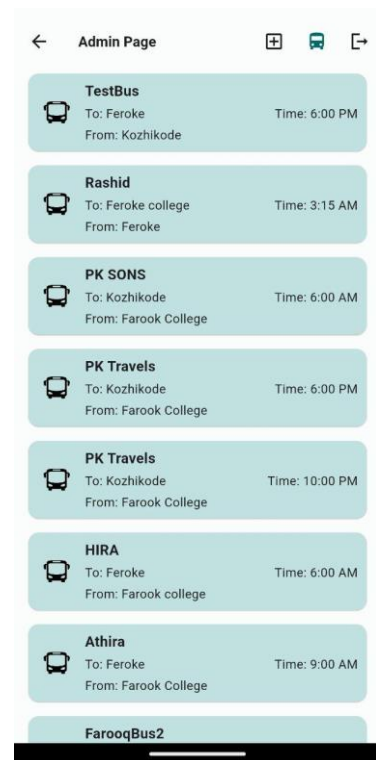
User Location



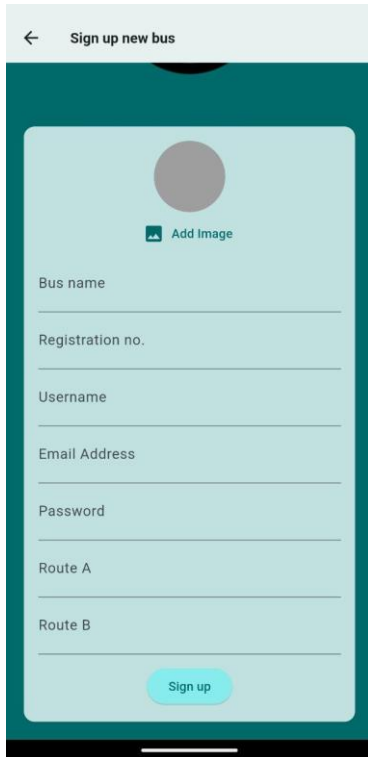
Log In Page of Special User & Admin



Admin Page

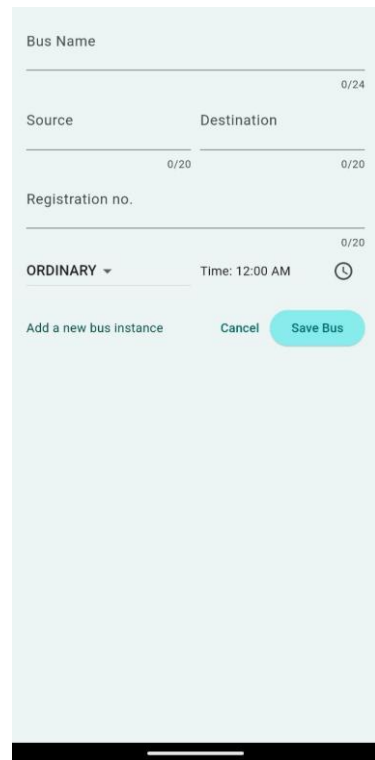


Sign up New Bus



A mobile app screen titled "Sign up new bus" with a back arrow. The form includes a profile picture placeholder with an "Add Image" button, and input fields for "Bus name", "Registration no.", "Username", "Email Address", "Password", "Route A", and "Route B". A "Sign up" button is at the bottom.

Add New Bus



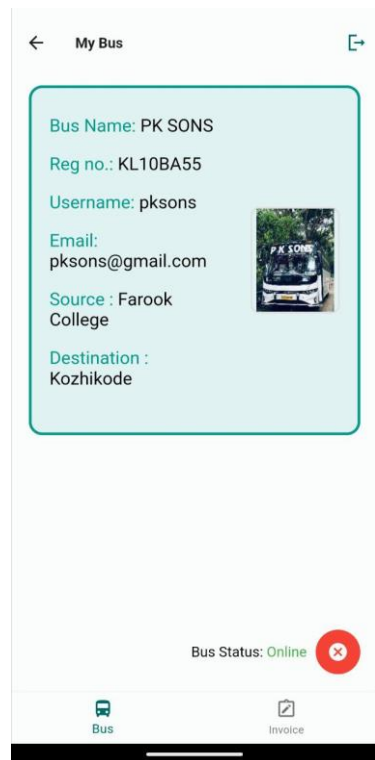
A mobile app screen titled "Add New Bus". It features input fields for "Bus Name", "Source", and "Destination", each with a character count (0/24, 0/20, 0/20). Below these is a "Registration no." field (0/20). A dropdown menu is set to "ORDINARY" and a time picker shows "Time: 12:00 AM". At the bottom are buttons for "Add a new bus instance", "Cancel", and "Save Bus".

Time



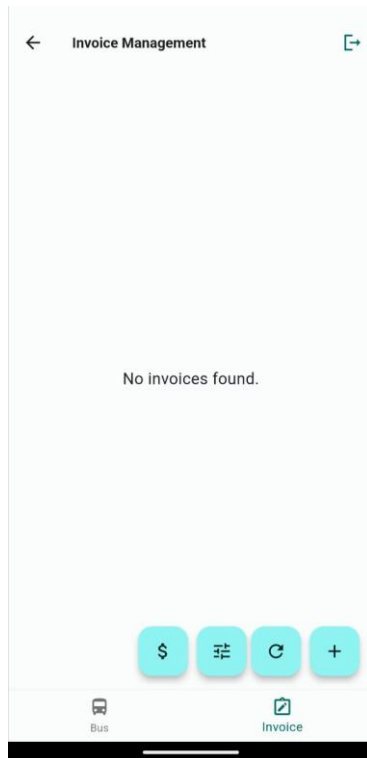
A time selection modal titled "Select time". It shows a digital clock set to "12 : 00" with "AM" and "PM" buttons. Below is an analog clock face with the hour hand pointing to 12. "Cancel" and "OK" buttons are at the bottom.

Bus Details

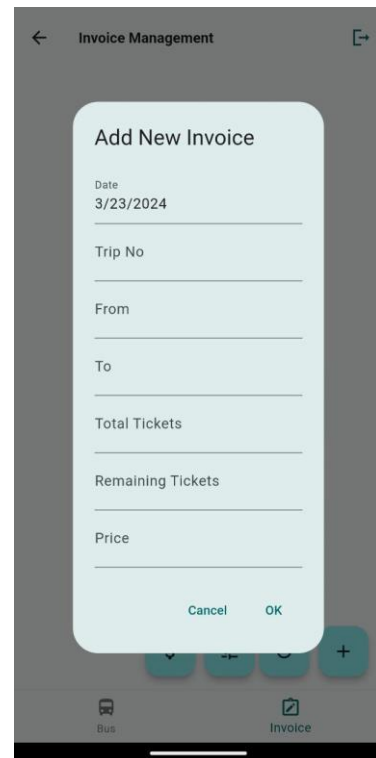


A mobile app screen titled "My Bus" with a back arrow and a share icon. It displays bus details in a light blue box: "Bus Name: PK SONS", "Reg no.: KL10BA55", "Username: pksons", "Email: pksons@gmail.com", "Source : Farook College", and "Destination : Kozhikode". A small image of the bus is shown next to the email. At the bottom right, it says "Bus Status: Online" with a red "X" icon. The bottom navigation bar has "Bus" and "Invoice" icons.

Invoice Management



Add New Invoice



CODE

Main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

import 'package:bms_sample/Default_page/buses.dart';

var kLightTealColorScheme = ColorScheme.fromSeed(
  seedColor: const Color.fromARGB(255, 0, 156, 156), // Light teal seed color
);

var kDarkColorScheme = ColorScheme.fromSeed(
  brightness: Brightness.dark,
  seedColor: const Color.fromRGBO(0, 53, 53, 1),
);

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(
    MaterialApp(
      theme: ThemeData.from(
        colorScheme: kLightTealColorScheme,
      ).copyWith(
        useMaterial3: true,
        scaffoldBackgroundColor:
          kLightTealColorScheme.background, // Light teal background
        appBarTheme: const AppBarTheme().copyWith(
          backgroundColor: kLightTealColorScheme.surface,
          foregroundColor: kLightTealColorScheme.onSurface,
        ),
        cardTheme: const CardTheme().copyWith(
          color: kLightTealColorScheme.secondaryContainer,
          margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
        ),
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: kLightTealColorScheme.primaryContainer,
          ),
        ),
      ),
    ),
  );
}
```

```

    ),
    textTheme: ThemeData().textTheme.copyWith(
      titleLarge: TextStyle(
        fontWeight: FontWeight.bold,
        color: kLightTealColorScheme.onSecondaryContainer,
        fontSize: 16,
      ),
    ),
  ),
  darkTheme: ThemeData.dark().copyWith(
    useMaterial3: true,
    colorScheme: kDarkColorScheme,
    // ... other dark theme customizations
  ),
  themeMode: ThemeMode.system, //default
  debugShowCheckedModeBanner: false,
  home: Buses(onSelectScreen: 'Bus Management System'),
),
);
}

```

Home Page

```

class Buses extends StatefulWidget {
  const Buses({ super.key, required this.onSelectScreen, });
  final String onSelectScreen;
  @override
  State<Buses> createState() => _BusesState();
}

class _BusesState extends State<Buses> {
  final _fromController = TextEditingController();
  final _toController = TextEditingController();
  bool _noBusFound = false;
  bool _isLoading = false;
  final _firestore = FirebaseFirestore.instance;
  int _selectedIndex = 0;

  void _onItemTapped(int index) {
    setState(() {
      _selectedIndex = index;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(

```

```

title: const Text('Bus Management System'),
actions: [
  IconButton(
    icon: const Icon(Icons.account_circle_outlined),
    onPressed: () {
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => StreamBuilder(
            stream: FirebaseAuth.instance.authStateChanges(),
            builder: (context, snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return const SplashScreen();
              }
              if (snapshot.hasData && AdminEmail == false && AdminPass == false) {
                return const MyBus();
              }
              if (snapshot.hasData && AdminEmail == true && AdminPass == true) {
                return const AdminPage();
              }
              return const AuthScreen();
            },
          ),
        ),
      );
    },
  ),
],
body: Column(
  children: [
    Padding(
      padding: const EdgeInsets.all(16.0),
      child: DecoratedBox(
        decoration: BoxDecoration(
          color: const Color.fromARGB(255, 156, 175, 173),
          borderRadius: BorderRadius.circular(8),
        ),
        child: Padding(
          padding: const EdgeInsets.all(8),
          child: Column(
            children: [
              Padding(
                padding: const EdgeInsets.symmetric(vertical: 4, horizontal: 6),
                child: TextFormField(
                  decoration: const InputDecoration(labelText: 'From'),
                  controller: _fromController,

```

```

    ),
  ),
  const SizedBox(height: 4),
  Padding(
    padding: const EdgeInsets.symmetric(vertical: 4, horizontal: 6),
    child: TextFormField(
      decoration: const InputDecoration(labelText: 'To'),
      controller: _toController,
    ),
  ),
  _isLoading ? const Center(child: CircularProgressIndicator()) : SizedBox(),
  _noBusFound ? const Center(
    child: Text(
      'No Bus found',
      style: TextStyle(color: Colors.red),
    ),
  ) : SizedBox(),
  const SizedBox(height: 8),
  SizedBox(
    width: MediaQuery.of(context).size.width * 0.9,
    child: ElevatedButton(
      onPressed: () async {
        String from = _fromController.text;
        String to = _toController.text;
        if (from.isEmpty || to.isEmpty) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
              content: Text('Please enter both origin and destination.'),
            ),
          );
        }
        return;
      }
    ),
    setState(() {
      _isLoading = true;
      _noBusFound = false;
    });
    QuerySnapshot<Map<String, dynamic>> snapshot = await _firestore
      .collection('Bus')
      .where('route_AB', isEqualTo: from)
      .where('route_BA', isEqualTo: to)
      .get();
    setState(() {
      _isLoading = false;
      if (snapshot.docs.isNotEmpty) {
        print('Bus found');
      } else {
        _noBusFound = true;
      }
    });
  ),

```

```

    }
  });
  if (snapshot.docs.isNotEmpty) {
    for (QueryDocumentSnapshot<Map<String, dynamic>> doc in
snapshot.docs) {
      Bus bus = Bus.fromMap(doc.data());
      showBusCard(context, bus);
      print('Bus name: ${bus.bus_name}');
      print('Bustype: ${bus.bustype}');
      print('Route from: ${bus.route_AB}');
      print('Route to: ${bus.route_BA}');
      print('time: ${bus.time}');
    }
  } else {
    print('No buses found for route $from - $to');
    setState(() {
      _noBusFound = true;
    });
    Future.delayed(const Duration(seconds: 3), () {
      setState(() {
        _noBusFound = false;
      });
    });
  }
},
child: Text('Find Bus'),
style: ElevatedButton.styleFrom(
  backgroundColor: Color.fromARGB(255, 8, 51, 9),
  foregroundColor: Colors.teal[400],
),
),
),
],
),
),
),
),
),
),
],
),
drawer: Drawer(
  child: ListView(
    padding: EdgeInsets.zero,
    children: [
      DrawerHeader(
        padding: const EdgeInsets.all(20),
        decoration: BoxDecoration(
          gradient: LinearGradient(colors: [

```

```

Theme.of(context).colorScheme.primaryContainer,
Theme.of(context)
  .colorScheme
  .primaryContainer
  .withOpacity(0.8),
], begin: Alignment.topLeft, end: Alignment.bottomRight),
),
child: Row(
  children: [
    Icon(
      Icons.directions_bus,
      size: 64,
      color: Theme.of(context).colorScheme.primary,
    ),
    const SizedBox(
      width: 18,
    ),
    Text(
      'NextBus',
      style: Theme.of(context).textTheme.titleLarge!.copyWith(
        color: Theme.of(context).colorScheme.primary,
        fontSize: 24,
      ),
    ),
  ],
),
),
ListTile(
  leading: Icon(
    Icons.access_time,
    size: 26,
    color: Theme.of(context).colorScheme.onBackground,
  ),
  title: Text(
    'Bus Schedule',
    style: Theme.of(context).textTheme.titleSmall!.copyWith(
      color: Theme.of(context).colorScheme.onBackground,
      fontSize: 18,
    ),
  ),
),
selected: _selectedIndex == 0,
onTap: () {
  // Update the state of the app
  _onItemTapped(0);
  // Then close the drawer
  Navigator.pop(context);
},

```

```

),
ListTile(
  leading: Icon(
    Icons.map_outlined,
    size: 26,
    color: Theme.of(context).colorScheme.onBackground,
  ),
  title: Text(
    'Map',
    style: Theme.of(context).textTheme.titleSmall!.copyWith(
      color: Theme.of(context).colorScheme.onBackground,
      fontSize: 18,
    ),
  ),
  selected: _selectedIndex == 1,
  onTap: () {
    // Update the state of the app
    _onItemTapped(1);
    // Then close the drawer
    Navigator.pop(context);
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => const MyLocationMap(),
      ),
    );
  },
),
ListTile(
  leading: Icon(
    Icons.help_outline_outlined,
    size: 26,
    color: Theme.of(context).colorScheme.onBackground,
  ),
  title: Text(
    'Help',
    style: Theme.of(context).textTheme.titleSmall!.copyWith(
      color: Theme.of(context).colorScheme.onBackground,
      fontSize: 18,
    ),
  ),
  selected: _selectedIndex == 3,
  onTap: () {
    // Update the state of the app
    _onItemTapped(3);
    // Then close the drawer
    Navigator.pop(context);
  },
),

```

```

Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const HelpPage(),
  ),
);
},
),
ListTile(
  leading: Icon(
    Icons.info_outline,
    size: 26,
    color: Theme.of(context).colorScheme.onBackground,
  ),
  title: Text(
    'About',
    style: Theme.of(context).textTheme.titleSmall!.copyWith(
      color: Theme.of(context).colorScheme.onBackground,
      fontSize: 18,
    ),
  ),
  selected: _selectedIndex == 2,
  onTap: () {
    // Update the state of the app
    _onItemTapped(2);
    // Then close the drawer
    Navigator.pop(context);
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => const AboutPage(),
      ),
    );
  },
),
],
),
),
);
}
}

```

```

Future<void> showBusCard(BuildContext context, Bus bus) async {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Available Bus'),

```

```

content: SingleChildScrollView(
  child: Card(
    margin: const EdgeInsets.all(8),
    child: Padding(
      padding: const EdgeInsets.all(8),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            crossAxisAlignment: CrossAxisAlignment.center,
            children: [
              Expanded(
                flex: 1,
                child: Align(
                  alignment: Alignment.centerLeft,
                  child: BustypeIcons[bus.bustype] ??
Image.asset('assets/images/default_bus.png'),
                ),
              ),
              const SizedBox(width: 10),
              Expanded(
                flex: 7,
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Row(
                      mainAxisAlignment: MainAxisAlignment.spaceBetween,
                      children: [
                        Text(
                          bus.bus_name,
                          style: const TextStyle(
                            fontWeight: FontWeight.bold,
                            fontSize: 16,
                            overflow: TextOverflow.ellipsis,
                          ),
                        ),
                        Text(
                          // 'Time: ',
                          '${bus.time}',
                          style: const TextStyle(fontSize: 14),
                        ),
                      ],
                    ),
                    const SizedBox(height: 4),
                    Text('To: ${bus.route_BA}', style: const TextStyle(fontSize: 14)),
                    const SizedBox(height: 4),
                    Row(

```

```

mainAxisAlignment: MainAxisAlignment.spaceBetween,
children: [
  Text('From: ${bus.route_AB}', style: const TextStyle(fontSize: 14)),
  const SizedBox(width: 10),
  ElevatedButton(
    onPressed: () async {
      Map<String, double> locationData = await fetchLatLon(bus.Regno);
      showDialog(
        context: context,
        builder: (context) => Center(
          child: AlertDialog(
            title: const Text('Location Data'),
            content: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              mainAxisAlignment: MainAxisAlignment.min, // Adjusts height based on content
              children: [
                Text('Latitude: ${locationData['latitude']}'),
                const SizedBox(height: 8), // Adjust spacing as needed
                Text('Longitude: ${locationData['longitude']}'),
              ],
            ),
            actions: [
              ElevatedButton( // Use TextButton for a flatter button style
                onPressed: () {
                  Navigator.push(context, MaterialPageRoute(builder: (context) =>
                    TrackBusPage(
                      latitude value: locationData['latitude']?.toDouble() ?? 0.0, // Pass
                      longitude value: locationData['longitude']?.toDouble() ?? 0.0, // Pass
                    )),
                ),
            ],
          ),
        ),
      );
    },
    child: const Text('Map'),
  ),
],
),
);
},
child: Text('Track'),
style: ElevatedButton.styleFrom(
  backgroundColor: Colors.teal,
  foregroundColor: Colors.white,
),
),
],
),

```

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    FutureBuilder<bool>(
      future: BusService().getAvailability(bus.Regno), // Assuming BusService is
the class containing getAvailability
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return Text('Error: ${snapshot.error}');
        }
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Text('Availability: Loading...');
        } else {
          if (snapshot.hasData) {
            bool availability = snapshot.data!;
            return Text('Availability: ${availability ? "Yes" : "No"}');
          } else {
            return const Text('Error: Availability data unavailable');
          }
        }
      },
    ),
  ],
),
],
),
],
),
],
),
],
),
],
),
),
);
}
```

Track bus

```
class TrackBusPage extends StatefulWidget {
  final double latitude;
  final double longitude;
  const TrackBusPage({Key? key, required this.latitude, required this.longitude})
    : super(key: key);

  @override
```

```

_TrackBusPageState createState() => _TrackBusPageState();
}
class _TrackBusPageState extends State<TrackBusPage> {
  late latLng.LatLng initialCenter;
  @override
  void initState() {
    super.initState();
    initialCenter = latLng.LatLng(widget.latitude, widget.longitude);
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Bus Location'),
      ),
      body: Center(
        child: widget.latitude == 0.0 && widget.longitude == 0.0
          ? const Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Icon(Icons.location_off_outlined, size: 50.0, color: Colors.grey),
                SizedBox(height: 16.0),
                Text(
                  'No location data available',
                  style: TextStyle(color: Colors.grey, fontSize: 18.0),
                ),
              ],
            )
          : Stack(
              children: [
                FlutterMap(
                  options: MapOptions(
                    initialCenter: initialCenter, // Use initialCenter for dynamic updates
                    initialZoom: 13.0,
                  ),
                  children: [
                    TileLayer(
                      urlTemplate:
'https://api.mapbox.com/styles/v1/raashidmap/clt2s6qi000ld01qz7remhmr1/tiles/256/{z}/{x}/{y}@2x?access_token=pk.eyJ1ljoicmFhc2hpZG1hcClslmEiOiJjbHQycm16d2wxanlvMmlsb25kNmI0M213In0.3sL2VMG5KDjM7wfgcpuVnQ',
                      additionalOptions: {
                        'accessToken':
'pk.eyJ1ljoicmFhc2hpZG1hcClslmEiOiJjbHQycm16d2wxanlvMmlsb25kNmI0M213In0.3sL2VMG5KDjM7wfgcpuVnQ',
                        'id': 'mapbox.mapbox-streets-v8',
                      },
                    ),

```

```

MarkerLayer(
  markers: [
    Marker(
      child: const Icon(Icons.location_on, size: 50.0, color: Colors.red,),
      width: 80.0,
      height: 80.0,
      point: initialCenter,
    ),
  ],
),
],
),
],
),
],
),
),
);
}
}

```

Login Page

```

import 'package:flutter/material.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});
  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _usernameController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _isLoading = false;
  @override
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Login'),
      ),
      body: Center(
        child: Container(
          margin: const EdgeInsets.all(24), // Margin around the border
          padding: const EdgeInsets.all(16), // Padding inside the border
          decoration: BoxDecoration(
            border: Border.all(color: Colors.grey), // Border around the content
            borderRadius: BorderRadius.circular(8), // Rounded corners
          ),
          child: Column(

```

```

crossAxisAlignment: CrossAxisAlignment.center,
children: [
  const Icon(Icons.account_circle_outlined, size: 64), // Login icon
  const SizedBox(height: 24),
  const Text('Account Login'),
  const SizedBox(height: 24),
  TextField(
    controller: _usernameController,
    decoration: const InputDecoration(
      labelText: 'Username',
    ),
  ),
  const SizedBox(height: 16),
  TextField(
    controller: _passwordController,
    obscureText: true,
    decoration: const InputDecoration(
      labelText: 'Password',
    ),
  ),
  const SizedBox(height: 24),
  ElevatedButton(
    onPressed: _isLoading
      ? null // Disable button while loading
      : () async {
        setState(() {
          _isLoading = true;
        });

        // Simulate authentication
        final isAuthorized = await _authenticateUser(
          _usernameController.text, _passwordController.text);

        setState(() {
          _isLoading = false;
        });

        if (isAuthorized) {
          Navigator.pushNamed(context, '/content');
        } else {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
              content: Text('Invalid Username or Password')),
            );
        }
      },
    child: const Text('Login'),
  ),

```

```

        _isLoading
        ? const CircularProgressIndicator()
        : const SizedBox
            .shrink(),
    ],
  ),
),
);
}
Future<bool> _authenticateUser(String username, String password) async {
  return username == 'test' && password == 'password';
}
}

```

Authentication

```

final _firebase = FirebaseAuth.instance;

class AuthScreen extends StatefulWidget {
  const AuthScreen({super.key});

  @override
  State<AuthScreen> createState() {
    return _AuthScreenState();
  }
}

bool AdminEmail = false;
bool AdminPass = false;

class _AuthScreenState extends State<AuthScreen> {
  final _form = GlobalKey<FormState>();

  var _isLogin = true;
  var _enteredEmail = "";
  var _enteredPassword = "";
  File? _selectedImage;
  var _isAuthenticating = false;

  void _submit() async {
    final isValid = _form.currentState!.validate();

    if (!isValid || !_isLogin && _selectedImage == null) {
      return;
    }

    _form.currentState!.save();
  }
}

```

```

try {
  setState(() {
    _isAuthenticating = true;
  });
  if (_isLogin) {
    final userCredentials = await _firebase.signInWithEmailAndPassword(
      email: _enteredEmail, password: _enteredPassword);
  } else {
    final userCredentials = await _firebase.createUserWithEmailAndPassword(
      email: _enteredEmail, password: _enteredPassword);
  }
} on FirebaseAuthException catch (error) {
  if (error.code == 'email-already-in-use') {
    // ...
  }
  ScaffoldMessenger.of(context).clearSnackBars();
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(error.message ?? 'Authentication failed.'),
    ),
  );
  setState(() {
    _isAuthenticating = false;
  });
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Log in'),
    ),
    backgroundColor: Theme.of(context).colorScheme.primary,
    body: Center(
      child: SingleChildScrollView(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Container(
              margin: const EdgeInsets.only(
                top: 30,
                bottom: 20,
                left: 20,
                right: 20,
              ),
              width: 200,

```

```

    child: Image.asset('assets/images/login.webp'),
  ),
  Card(
    margin: const EdgeInsets.all(20),
    child: SingleChildScrollView(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _form,
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              TextFormField(
                decoration: const InputDecoration(
                  labelText: 'Email Address'),
                keyboardType: TextInputType.emailAddress,
                autocorrect: false,
                textCapitalization: TextCapitalization.none,
                validator: (value) {
                  if (value == null ||
                      value.trim().isEmpty ||
                      !value.contains('@')) {
                    return 'Please enter a valid email address.';
                  }
                  return null;
                },
              ),
              onSaved: (value) {
                _enteredEmail = value!;
                if (value == 'FCfarooq@gmail.com') {
                  AdminEmail = true;
                } else {
                  AdminEmail = false;
                }
              },
            ],
          ),
        ),
      ),
    ),
    TextFormField(
      decoration:
        const InputDecoration(labelText: 'Password'),
      obscureText: true,
      validator: (value) {
        if (value == null || value.trim().length < 6) {
          return 'Password must be at least 6 characters long';
        }
        return null;
      },
      onSaved: (value) {
        _enteredPassword = value!;
        if (value == '0987654') {

```

```

        AdminPass = true;
      } else {
        AdminPass = false;
      }
    },
  ),
const SizedBox(
  height: 12,
),
if (_isAuthenticating)
  const CircularProgressIndicator(),
if (!_isAuthenticating)
  ElevatedButton(
    onPressed: _submit,
    style: ElevatedButton.styleFrom(
      backgroundColor: Theme.of(context)
        .colorScheme
        .primaryContainer,
    ),
    child: Text(_isLogin ? 'Log in' : 'Sign up'),
  ),
],
),
),
),
),
),
),
],
),
),
),
);
}
}

```

Invoice

```

class MyInvoiceScreen extends StatefulWidget {
  const MyInvoiceScreen({super.key});
  @override
  State<MyInvoiceScreen> createState() => _MyInvoiceScreenState();
}
class MyFirestoreService {
  static Future<void> addInvoice(String tripNo, String from, String to,
    int totalTickets, int remainingTickets, double price, DateTime selectedDate,) async {
    // Get the currently accessed user
    final currentUser = FirebaseAuth.instance.currentUser;

```

```

if (currentUser != null) {
    // Get the user ID
    final userId = currentUser.uid;

    // Access the user's document in SPusers collection
    final docRef = FirebaseFirestore.instance.collection('SPusers').doc(userId);

    // Check for the existence of the "invoices" subcollection
    final hasInvoicesSubcollection =
        await docRef.collection('invoices').get().then((snapshot) =>
snapshot.docs.isNotEmpty());
    if (hasInvoicesSubcollection) {
        // Add invoice to the existing subcollection
        await docRef.collection('invoices').add({
            'tripNo': tripNo,
            'from': from,
            'to': to,
            'totalTickets': totalTickets,
            'remainingTickets': remainingTickets,
            'price': price,
            'date': selectedDate,
        });
    } else {
        // Create the "invoices" subcollection and add the first invoice
        await docRef.collection('invoices').add({
            'tripNo': tripNo,
            'from': from,
            'to': to,
            'totalTickets': totalTickets,
            'remainingTickets': remainingTickets,
            'price': price,
            'date': selectedDate,
        });
    }
} else {
    // Handle the case where no user is signed in
    debugPrint('No user signed in, cannot add invoice to Firestore');
}
}

static Future<void> deleteInvoice(String invoiceId) async {
    final currentUser = FirebaseAuth.instance.currentUser;
    if (currentUser != null) {
        await FirebaseFirestore.instance
            .collection('SPusers')
            .doc(currentUser.uid)
            .collection('invoices')
            .doc(invoiceId)
            .delete();
    }
}

```

```

    } else {
        debugPrint('No user signed in, cannot delete invoice');
    }
}
}

class _MyInvoiceScreenState extends State<MyInvoiceScreen> {
    List<Invoice> invoices = [];
    DateTime selectedDate = DateTime.now();
    // **Create an instance of FirebaseAuth**
    final FirebaseAuth auth = FirebaseAuth.instance;
    final firestore = FirebaseFirestore.instance;

    Future<void> _fetchInvoices() async {
        final user = auth.currentUser; // Use the created 'auth' instance
        if (user != null) {
            try {
                final invoiceRef = firestore.collection('SPusers').doc(user.uid).collection('invoices');
                final snapshot = await invoiceRef.orderBy('tripNo', descending: true).get();
                final fetchedInvoices = snapshot.docs
                    .map((doc) => Invoice.fromMap(doc.data()))
                    .toList();
                setState(() {
                    invoices = fetchedInvoices;
                });
            } catch (error) {
                print('Error fetching invoices: $error');
            }
        } else {
            debugPrint('No user signed in, cannot fetch invoices');
        }
    }

    Future<void> _deleteInvoice(String invoiceId) async {
        try {
            await MyFirestoreService.deleteInvoice(invoiceId);
            setState(() {
                invoices.removeWhere((invoice) => invoice.id == invoiceId);
            });
            ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                    content: Text('Invoice deleted successfully'),
                ),
            );
        } catch (error) {
            debugPrint('Error deleting invoice: $error');
            ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                    content: Text('Failed to delete invoice'),
                ),
            );
        }
    }
}

```

```

    ),
  );
}
}

```

```

  @override
void initState() {
  super.initState();
  _fetchInvoices();
}

```

```

  @override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: invoices.isEmpty
        ? const Text("No invoices found.", style: TextStyle(fontSize: 18))
        : ListView.builder(
            itemCount: invoices.length,
            itemBuilder: (context, index) {
              return Dismissible(
                key: UniqueKey(),
                direction: DismissDirection.endToStart,
                onDismissed: (direction) {
                  final dismissedInvoice = invoices.removeAt(index);
                  ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(
                      content: Text('Invoice deleted'),
                      action: SnackBarAction(
                        label: 'Undo',
                        onPressed: () {
                          setState(() {
                            invoices.insert(index, dismissedInvoice);
                          });
                        },
                      ),
                    ),
                  );
                  _deleteInvoice(dismissedInvoice.id);
                },
                background: Container(
                  color: Colors.red,
                  padding: const EdgeInsets.all(16),
                  child: const Icon(
                    Icons.delete,
                    color: Colors.white,
                  ),
                ),
              );
            },
          ),
    ),
  );
}

```

```

        child: Card(
          child: SingleChildScrollView(
            child: ListTile(
              title: Text("Trip No: ${invoices[index].tripNo}"),
              subtitle: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text("Date: ${DateFormat.yMd().format(invoices[index].date)}"),
                  Text("From: ${invoices[index].from} - To: ${invoices[index].to}"),
                  Text("Income: ${((invoices[index].totalTickets -
invoices[index].remainingTickets) * invoices[index].price)}"),
                ],
              ),
            ),
          ),
        ),
      );
    },
  ),
),
floatingActionButton: FloatingActionButton(
  onPressed: _showAddInvoiceDialog,
  child: const Icon(Icons.add),
),
);
}

```

```

void _showAddInvoiceDialog() {
  String tripNo = "";
  String from = "";
  String to = "";
  int totalTickets = 0;
  int remainingTickets = 0;
  double price = 0.0;
  // DateTime selectedDate = ;

```

```

showDialog(
  context: context,
  builder: (BuildContext context) {
    return AlertDialog(
      title: const Text("Add New Invoice"),
      content: SingleChildScrollView(
        child: Column(
          children: [
            TextField(
              decoration: const InputDecoration(labelText: 'Trip No'),
              keyboardType: TextInputType.number,
              onChanged: (value) => tripNo = value,

```

```

    ),
    TextField(
      decoration: const InputDecoration(labelText: 'From'),
      onChanged: (value) => from = value,
    ),
    TextField(
      decoration: const InputDecoration(labelText: 'To'),
      onChanged: (value) => to = value,
    ),
    TextField(
      decoration: const InputDecoration(labelText: 'Total Tickets'),
      keyboardType: TextInputType.number,
      onChanged: (value) => totalTickets = int.tryParse(value) ?? 0,
    ),
    TextField(
      decoration: const InputDecoration(labelText: 'Remaining Tickets'),
      keyboardType: TextInputType.number,
      onChanged: (value) => remainingTickets = int.tryParse(value) ?? 0,
    ),
    TextField(
      decoration: const InputDecoration(labelText: 'Price'),
      keyboardType: TextInputType.number,
      onChanged: (value) => price = double.tryParse(value) ?? 0.0,
    ),
  ],
),
),
actions: [
  TextButton(
    onPressed: () {
      Navigator.of(context).pop();
    },
    child: const Text('Cancel'),
  ),
  TextButton(
    onPressed: () async {
      if (tripNo.isNotEmpty &&
        from.isNotEmpty &&
        to.isNotEmpty &&
        totalTickets > 0 &&
        remainingTickets >= 0 &&
        remainingTickets <= totalTickets &&
        price > 0.0 ) {
        // Call Firestore service to add data
        await MyFirestoreService.addInvoice(
          tripNo, from, to, totalTickets, remainingTickets, price, selectedDate, );
        setState(() {
          invoices.add(Invoice(

```

```

        tripNo: tripNo,
        from: from,
        to: to,
        totalTickets: totalTickets,
        remainingTickets: remainingTickets,
        price: price,
        date: selectedDate,
    ));
});
Navigator.of(context).pop();
} else {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Please fill in all fields correctly.'),
    ),
  );
}
},
child: const Text('OK'),
),
],
);
},
);
}
}

```

Admin

// Constants

```
final Uuid uuid = Uuid();
```

```
enum Bustype { Ordinary, SF, FP, mini }
```

```
final BustypeIcons = {
  Bustype.Ordinary: Image.asset('assets/images/ordinary_black.png'),
  Bustype.SF: Image.asset('assets/images/SF_black.png'),
  Bustype.FP: Image.asset('assets/images/FP_black.png'),
  Bustype.mini: Image.asset('assets/images/mini_black.png'),
};
```

```
class Bus {
  final String id; // Unique identifier for each bus
  final String bus_name;
  final String route_AB;
  final String route_BA;
  final Bustype bustype;
```

```

final String Regno;
final String time;

// Constructor to create a new Bus
Bus({
  this.id = "",
  required this.bus_name,
  required this.route_AB,
  required this.route_BA,
  required this.bustype,
  required this.Regno,
  required this.time,
});

// Constructor to create a Bus from a Firestore document
factory Bus.fromMap(Map<String, dynamic> data) {
  // Handle potential errors here, e.g., check for required fields
  // Check for 'id' and generate if missing
  final String id = data['id'] as String? ?? uuid.v4();

  return Bus(
    id: id,
    bus_name: data['bus_name'] as String,
    route_AB: data['route_AB'] as String,
    route_BA: data['route_BA'] as String,
    Regno: data['Regno'] as String,
    bustype: Bustype.values.firstWhere((type) => type.toString() == data['bustype'], orElse: ()
=> Bustype.SF),
    time: data['time'] as String,
  );
}

@override
String toString() {
  return 'Bus(id: $id, bus_name: $bus_name, route_AB: $route_AB, route_BA: $route_BA,
bustype: $bustype, time: $time,)'
}
}

```